Installation

Python 2.7 is a pre-requisite for installing Volatility. To install on a Linux system, you can download and extract the archive from *https://github.com/volatilityfoundation/volatility*.

Then run the following command:

```
sudo python setup.py install
```

Or, run the following command:

```
apt-get install volatility
```

Installation of Volatility in Windows is manual and needs additional packages. These are listed below.

Distorm 3: A powerful disassembler library for x86/AMD64

Yara: A malware identification and classification tool

Pycrypto: The Python cryptography toolkit

Pillow: The Python imaging library

Openpyxl: The Python library to read/write Excel

Ujson: An ultra-fast JSON parsing library

Pytz: This is for time zone conversion

Installation using an executable

A standalone executable can be downloaded from *http://www.volatilityfoundation.org/#!24/c12wa* (available in both the standalone version and the Python Win32 module version).

Memory inspection

In order to analyse the memory dumps, the profile should be defined prior to execution. You can start trying the software using the memory sample available for testing purposes at *https://github.com/volatilityfoundation/volatility/wiki/Memory-Samples*.

The syntax is:

```
python vol.py -h
```

The above executed command shows you the available options in the Volatility framework.

Figure 1: Screenshot of the Volatility framework

In general, everything in the OS traverses RAM. Tracing the RAM fingerprints can be helpful to detect some advanced malware. In this article, we explore some plugins, which are available in the Volatility framework. Let me take a memory sample of the malware 'stuxnet' downloaded from the above link. Unzip the *Stuxnet.zip* file and extract *Stuxnet.vmem*, which is a virtual memory file. Given below are a few fingerprints that have been extracted from the sample. Volatility helps us to identify the OS' profile information, which gives the meta information about the memory file. Volatility can inspect the live memory image of any operating system. The framework can give the status of an active process, a hidden process, unlinked processes, DLL loaded in runtime, socket information, external connection information, etc. Some of the plugins were tested and the results were given.

1. *Imageinfo*: This identifies the profile image. The syntax is:

```
python vol.py imageinfo -f Stuxnet.vmem
```

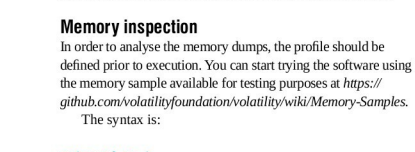

Figure 2: Screenshot of the *Imageinfo* plugin which gives the suggested profiles

2. *Pslist:* This lists the running process. The syntax is:

```
python vol.py pslist -f Stuxnet.vmem
```


Figure 3: Screenshot of the *pslist* plugin which lists the running processes

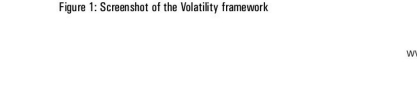

Figure 4: Screenshot of the *psscan* plugin which scans for hidden processes